

Rising Waters: A Machine Learning Approach to Flood Prediction

Project Description:

Rising Waters: A Machine Learning Approach to Flood Prediction is an intelligent web-based application that predicts the likelihood of flood occurrence using historical rainfall and meteorological data. The project aims to improve disaster preparedness by providing accurate and timely flood risk predictions, enabling authorities and communities to take preventive actions before flood events occur.

The system utilizes historical weather parameters such as annual rainfall, seasonal rainfall, cloud cover, and other environmental factors to train multiple machine learning classification models, including Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost. After evaluating the models based on performance metrics such as accuracy, precision, recall, and F1-score, the best-performing model is selected and integrated into a Flask web application.

The web application provides an intuitive interface where users can enter rainfall and weather-related information to receive instant flood risk predictions. The application performs data preprocessing, feature scaling, and prediction using the trained machine learning model, then displays whether the given conditions indicate a Flood Risk or No Flood Risk.

By combining data analysis, machine learning, and web technologies, the project offers a scalable and user-friendly solution for supporting disaster management authorities, meteorologists, and residents in making informed decisions. Future enhancements may include integration with real-time weather APIs, GIS-based flood mapping, and automated alert systems to further improve prediction accuracy and emergency response.

Scenarios

Scenario 1: Early Flood Warning and Evacuation Planning

A **meteorologist** monitors weather conditions in a flood-prone district during the monsoon season. After entering current rainfall and meteorological data into the Flood Prediction System, the machine learning model predicts a **high probability of flooding**. Based on this prediction, disaster management authorities issue early warnings, initiate evacuation plans, and alert residents to move to safer locations before the flood occurs.

Scenario 2: Disaster Response and Resource Allocation

A **disaster management officer** uses the web application to monitor flood risks across multiple regions. By entering rainfall and weather information for different locations, the

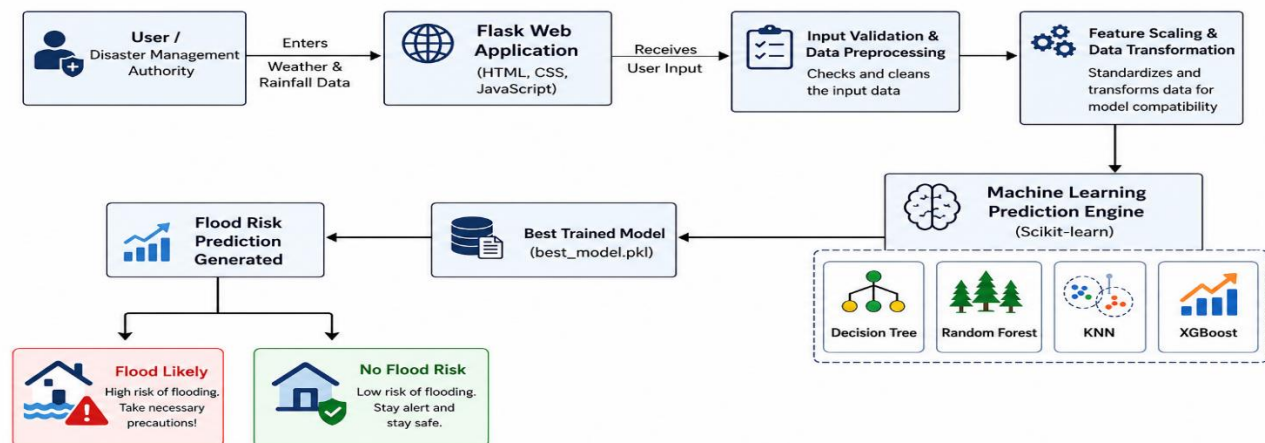
system provides instant flood risk predictions. The officer uses these predictions to prioritize the deployment of emergency teams, rescue equipment, medical supplies, and relief materials to high-risk areas, ensuring an effective and timely disaster response.

Scenario 3: Community Awareness and Personal Safety

A **resident living in a flood-prone area** accesses the Flood Prediction System through the web application to check the likelihood of flooding in their locality. After entering the latest rainfall and weather parameters, the application predicts the flood risk. If a high flood risk is detected, the resident takes precautionary measures such as protecting valuables, preparing emergency supplies, and relocating family members to a safer place.

Technical Architecture:

Technical Architecture:



Rising Waters: A Machine Learning Approach to Flood Prediction follows a modular three-layer architecture to provide accurate flood risk predictions using historical rainfall and weather data. The frontend is developed using HTML, CSS, and JavaScript, providing an intuitive interface for users to enter weather-related parameters. The backend is implemented using the Flask framework, which manages request handling, input validation, and communication with the machine learning prediction engine.

The prediction engine preprocesses the input data, applies feature scaling, and evaluates it using trained classification models such as Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost. The best-performing model, stored as `best_model.pkl`, generates the flood prediction, which is displayed through the web application. The application can be deployed on platforms such as Render or IBM Cloud, making it accessible for disaster management authorities, meteorologists, and the public.

Note: This project does not use a relational database, so an Entity Relationship Diagram (ERD) is not required.

Pre-requisites

- Python Programming: Python 3.10 or above
- Flask Framework: Knowledge of Flask for developing the web application
- Machine Learning: Understanding of classification algorithms (Decision Tree, Random Forest, KNN, XGBoost)
- Data Analysis: Experience with Pandas and NumPy for data preprocessing
- Visualization: Matplotlib and Seaborn for Exploratory Data Analysis (EDA)
- Frontend Development: HTML, CSS, and JavaScript for building the user interface
- Version Control: Git and GitHub for source code management
- Development Environment: Visual Studio Code or Jupyter Notebook
- Deployment Platform: Render or IBM Cloud for hosting the Flask application

Project Workflow:

Milestone 1: Dataset Collection and System Architecture

This milestone focuses on collecting historical rainfall and flood datasets, selecting suitable machine learning algorithms, designing the overall system architecture, and preparing the development environment. The objective is to build a strong foundation for developing an intelligent flood prediction system capable of accurately predicting flood risks using historical weather and rainfall data.

Milestone 2: Data Preprocessing and Model Development

This milestone involves preparing the collected datasets for machine learning by performing data cleaning, exploratory data analysis (EDA), feature engineering, and feature scaling. Multiple classification algorithms are trained and evaluated to identify the most accurate model for flood prediction.

- **Activity 2.1:** Perform data preprocessing by handling missing values, removing duplicate records, detecting outliers, encoding categorical variables (if required), and applying feature scaling.
- **Activity 2.2:** Train and compare **Decision Tree**, **Random Forest**, **K-Nearest Neighbors (KNN)**, and **XGBoost** models using evaluation metrics such as Accuracy, Precision, Recall, F1-Score, and Confusion Matrix.
- **Activity 2.3:** Save the best-performing model (best_model.pkl) along with preprocessing files (scaler.pkl, feature_columns.pkl, and feature_stats.json) for deployment.

Milestone 3: Flask Application Development

- **Activity 3.1:** Develop the Flask application (app.py) by creating routes for the Home, Prediction, and Result pages.
- **Activity 3.2:** Integrate the trained machine learning model with the Flask backend to preprocess user inputs, perform flood prediction, and display prediction results.

Milestone 4: Frontend Development

- **Activity 4.1:** Design a responsive and user-friendly interface using **HTML**, **CSS**, and **JavaScript** for entering rainfall and weather parameters.
- **Activity 4.2:** Develop Flask templates (home.html, predict.html, result.html, flood.html, and no_flood.html) to display prediction results dynamically based on user inputs.

Milestone 5: Testing and Deployment

- **Activity 5.1:** Test the complete application locally by validating user inputs, verifying prediction accuracy, and ensuring proper integration between the frontend, Flask backend, and machine learning model.
- **Activity 5.2:** Deploy the Flask application on a cloud platform such as **Render** or **IBM Cloud**, making the Flood Prediction System publicly accessible for users.

Milestone 6: Conclusion

Evaluate the overall performance of the Flood Prediction System, summarize the project outcomes, and identify future enhancements such as integrating real-time weather APIs, GIS-based flood visualization, and automated alert notifications to improve disaster preparedness and decision-making.

Milestone 1: Dataset Collection and System Architecture

This milestone focuses on collecting historical rainfall and flood datasets, preparing the development environment, designing the overall system architecture, and selecting suitable machine learning algorithms. The objective is to establish a strong foundation for building an intelligent flood prediction system capable of accurately predicting flood risks from historical weather data.

Activity 1.1: Collect Flood Prediction Dataset

Before developing the prediction model, collect a reliable dataset containing historical rainfall and flood-related information.

Step 1 — Download the Dataset

Download historical rainfall and flood datasets from Kaggle or other open-source repositories.

Example datasets:

- Flood Dataset.xlsx
- Rainfall in India (1901–2015).xlsx

Step 2 — Verify Dataset Quality

Ensure the dataset contains important features such as:

- Annual Rainfall
- Seasonal Rainfall
- Cloud Cover
- Temperature
- Humidity
- River Water Level (if available)
- Flood Occurrence (Target Variable)

Activity 1.2: Perform Exploratory Data Analysis (EDA)

Before training the model:

- Check missing values
- Remove duplicate records
- Analyze rainfall distribution
- Detect outliers
- Generate Heatmaps
- Plot Correlation Matrix
- Visualize Feature Relationships

Activity 1.3: Design the System Architecture

Prepare the architecture showing:

Frontend

- HTML
- CSS

- JavaScript

Backend

- Flask

Machine Learning

- XGBoost
- Random Forest
- Decision Tree
- KNN

Storage

- Dataset
- Trained Models

Prediction Output

- Flood Risk
- No Flood Risk

Activity 1.4: Set Up the Development Environment

Install Python

```
python --version
```

Create Virtual Environment

```
python -m venv venv
```

Activate Environment

Windows

```
venv\Scripts\activate
```

Linux

```
source venv/bin/activate
```

Install Requirements

```
pip install -r requirements.txt
```

Run Flask

```
python app.py
```

Project Structure

rainfall-main/

|

|— app.py

|— requirements.txt

- └─ runtime.txt

- └─ Procfile

- └─ README.md

- |

- └─ dataset/

- | └─ flood_dataset.xlsx

- | └─ rainfall_in_india_1901_2015.xlsx

- |

- └─ models/

- | └─ best_model.pkl

- | └─ scaler.pkl

- | └─ feature_columns.pkl

- | └─ feature_stats.json

- |

- └─ notebooks/

- | └─ eda_and_modeling.ipynb

- |

- └─ static/

- | └─ css/

- | └─ js/

- | └─ images/

- |


```
|— templates/
| |— home.html
| |— predict.html
| |— result.html
| |— flood.html
| |— no_flood.html
|
|— utils/
|— explainers.py
```

Milestone 2: Data Preprocessing and Model Development

Activity 2.1: Data Preprocessing

Perform:

- Missing Value Handling
- Duplicate Removal
- Outlier Detection
- Feature Scaling
- Train-Test Split

Activity 2.2: Train Machine Learning Models

Train the following models:

- Decision Tree
- Random Forest
- K-Nearest Neighbors
- XGBoost

Compare using

- Accuracy
- Precision
- Recall
- F1 Score
- Confusion Matrix

Activity 2.3: Save the Best Model

After evaluation, save
best_model.pkl

scaler.pkl

feature_columns.pkl

feature_stats.json

using

```
joblib.dump(best_model,"models/best_model.pkl")
```

Milestone 3: Flask Application Development

This milestone focuses on integrating the trained machine learning model into a Flask web application, allowing users to enter rainfall and weather parameters and receive flood risk predictions.

Activity 3.1: Develop Flask Routes

Create routes:

/

Home Page

/predict

Prediction Page

/result

Prediction Result

Activity 3.2: Load Model

```
model = joblib.load("models/best_model.pkl")
```

Activity 3.3: Process User Input

Collect:

- Rainfall
- Cloud Visibility
- Seasonal Rainfall
- Temperature
- Humidity

Validate inputs.

Activity 3.4: Predict Flood Risk

Use

```
prediction = model.predict(input_data)
```

Display

High Flood Risk

or

Low Flood Risk

Milestone 4: Frontend Development

Design pages

- Home
- Prediction Form
- Flood Result
- No Flood Result

Develop using

- HTML
- CSS
- Bootstrap
- JavaScript

Templates

home.html

predict.html

result.html

flood.html

no_flood.html

Milestone 5: Testing and Deployment

Activity 5.1 Testing

Test

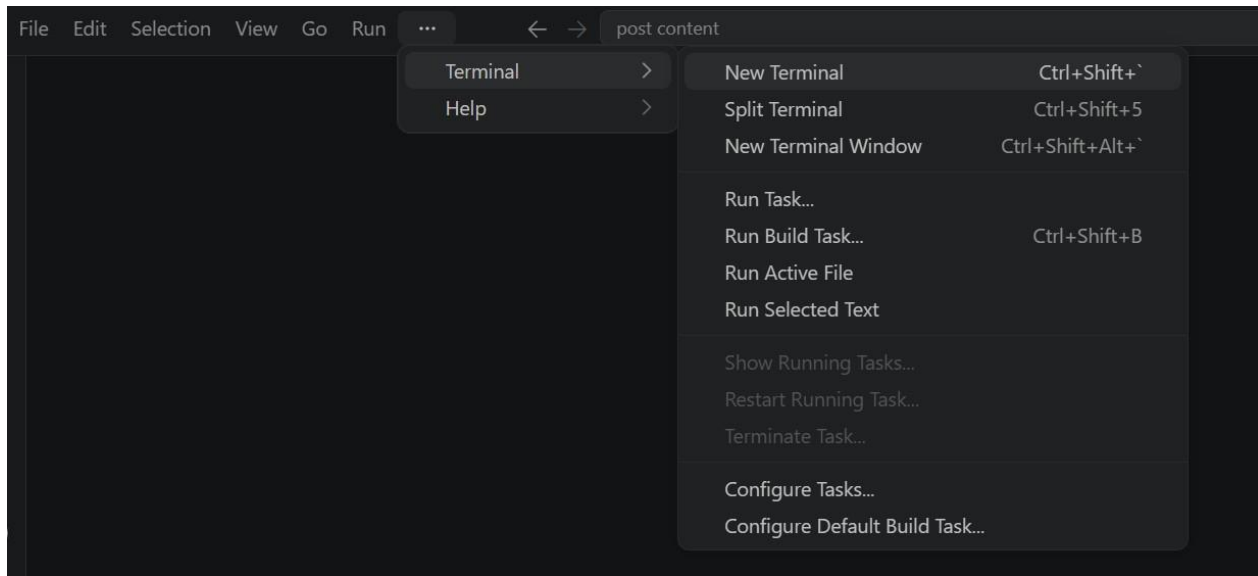
- ✓ Dataset Loading
- ✓ Model Prediction
- ✓ Flask Routes
- ✓ Input Validation

✓ Output Display

Activity 5.2 Deployment

Deploy on

- Render
- IBM Cloud



- Verify
 - Application loads successfully
 - Prediction works
 - Static files load correctly
-

Milestone 6: Project Completion

Evaluate:

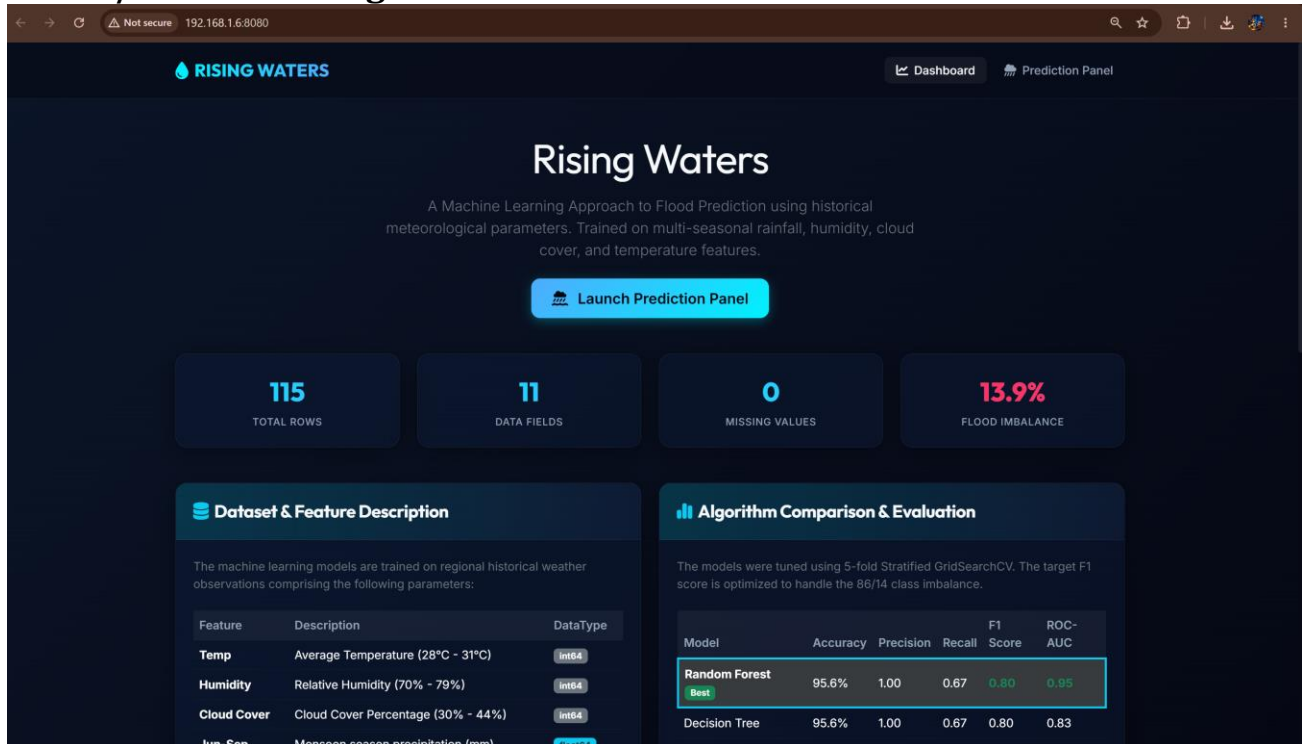
- Model Accuracy
- User Experience
- Prediction Speed

Future Enhancements

- Live Weather API Integration
- SMS Alert System
- GIS Flood Mapping
- Mobile Application
- Real-Time Rainfall Monitoring

Exploring the Website's Web Pages:

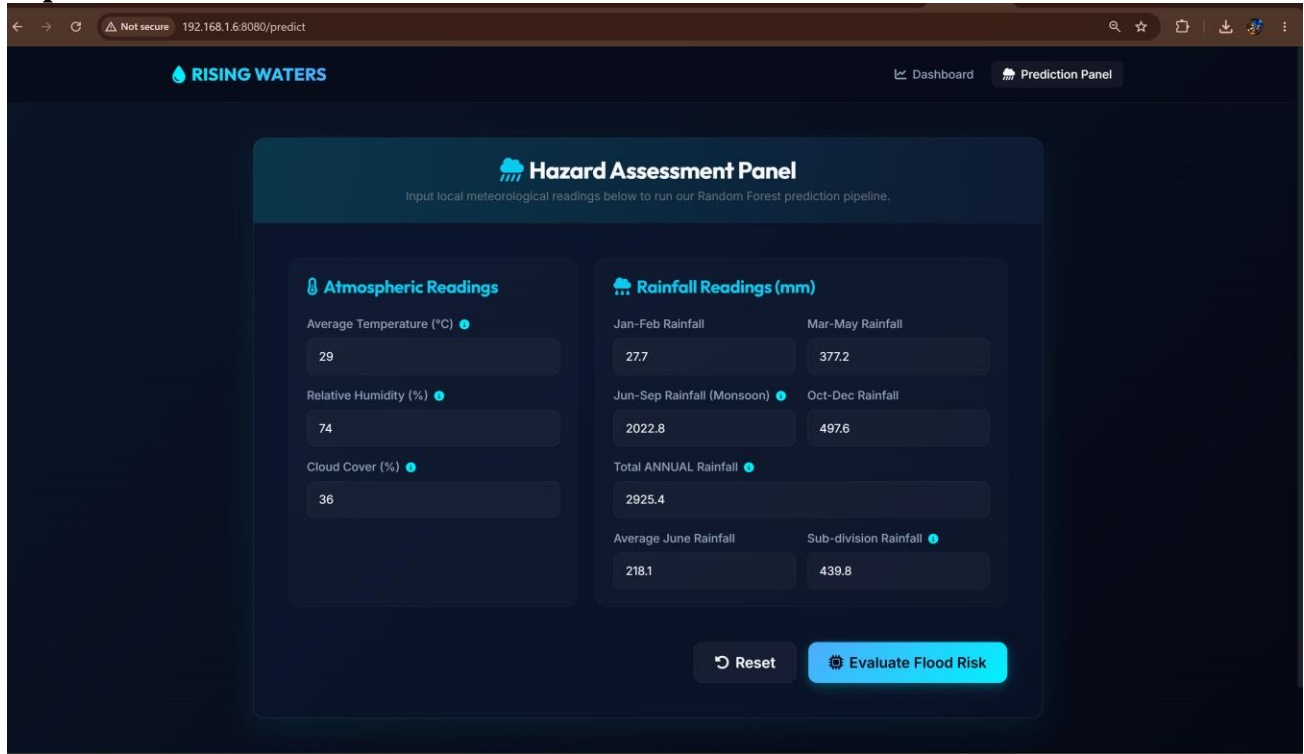
Home / Generator Page:



Description:

The Home Dashboard provides an overview of the Flood Prediction System. It displays project statistics, dataset information, and machine learning model performance. Users can launch the prediction panel to assess flood risk using weather and rainfall parameters.

Input Section:



The screenshot shows a web browser window with the URL `192.168.1.6:8080/predict`. The page has a dark theme and a navigation bar with "RISING WATERS" and "Prediction Panel" tabs. The main content area is titled "Hazard Assessment Panel" and includes a sub-header: "Input local meteorological readings below to run our Random Forest prediction pipeline."

The input section is divided into two columns:

- Atmospheric Readings:**
 - Average Temperature (°C): 29
 - Relative Humidity (%): 74
 - Cloud Cover (%): 36
- Rainfall Readings (mm):**
 - Jan-Feb Rainfall: 27.7
 - Mar-May Rainfall: 377.2
 - Jun-Sep Rainfall (Monsoon): 2022.8
 - Oct-Dec Rainfall: 497.6
 - Total ANNUAL Rainfall: 2925.4
 - Average June Rainfall: 218.1
 - Sub-division Rainfall: 439.8

At the bottom right, there are two buttons: "Reset" and "Evaluate Flood Risk".

Description:

The Hazard Assessment Panel allows users to enter meteorological and rainfall data such as temperature, humidity, cloud cover, seasonal rainfall, and annual rainfall. After submitting the values, the machine learning model analyzes the inputs and predicts the likelihood of a flood.

Results Section:



Description:

The Assessment Report displays the flood prediction result generated by the machine learning model. It shows the predicted flood risk level, confidence score, feature importance, and safety recommendations to help users understand the prediction and make informed decisions.

Conclusion:

The **Rising Waters: A Machine Learning Approach to Flood Prediction** project successfully demonstrates the use of Machine Learning and Flask to develop an intelligent flood prediction system based on historical rainfall and weather data. By applying data preprocessing, exploratory data analysis, feature engineering, and model evaluation techniques, multiple classification algorithms—including **Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost**—were trained and compared to identify the most accurate prediction model.

The selected model was integrated into a user-friendly Flask web application that enables users to enter weather-related parameters and receive instant flood risk predictions. This system can assist meteorologists, disaster management authorities, and local communities in making timely decisions, improving disaster preparedness, and reducing the impact of flood-related hazards.

Overall, the project highlights the practical application of machine learning in disaster management by providing a reliable, scalable, and accessible flood prediction solution. With future enhancements such as real-time weather API integration, GIS-based visualization, automated alert notifications, and mobile application support, the system can be further improved to deliver more accurate predictions and contribute to safer and more resilient communities.